

A Coherent Technical Synthesis of the Paper A *Two-Stage Prompt-Tuning Large Language Model as a Reranker for Conversational Recommender Systems*

Contents

1	Executive Summary	4
2	Problem Setting and Overall Objective	4
3	High-Level Summary of the Proposed Method	5
3.1	Stage 1: Pre-ranking	5
3.2	Stage 2: Re-ranking with Prompt-Tuned LLM	5
3.3	Stage 3: Response Generation	6
4	Complete Pipeline: Step-by-Step Explanation	6
4.1	Step 1: Receive the dialogue history	6
4.2	Step 2: Extract entities from the dialogue	6
4.3	Step 3: Build the External Knowledge Graph (EKG)	6
4.4	Step 4: Build the Conversational Knowledge Graph (CKG)	7
4.5	Step 5: Encode the dialogue text with RoBERTa	7
4.6	Step 6: Encode the external KG with R-GCN	7
4.7	Step 7: Encode the conversational KG with GCN	8
4.8	Step 8: Perform semantic fusion between dialogue and entities	8
4.9	Step 9: Compute a preliminary recommendation with DialogPT	9
4.10	Step 10: Re-rank candidates with a prompt-tuned LLM	9
4.11	Step 11: Generate the final response	10
5	Input Elements, Their Origin, and Their Transformation	10
5.1	Dialogue history	10
5.2	Dialogue entities	10
5.3	External KG	10
5.4	Conversational KG	11
5.5	Candidate shortlist	11
6	Experiments: Datasets, Evaluation, and Baselines	11
6.1	Datasets	11
6.1.1	INSPIRED	11
6.1.2	ReDial	12
6.2	What the paper reports	12

clearly	12	6.3 What the paper does not specify clearly	12
.	12	6.4 Baselines	12
12 6.4.1 Traditional recommendation baselines	12	6.4.2 LLM / general conversational baselines	13
13 6.4.3 Conversational recommender baselines	13	6.5 Evaluation metrics	13
.	13	6.5.1 Recommendation metrics	13
. 13 6.5.2 Dialogue generation metrics	13	6.5.3 Human evaluation	13
.	13	6.6 Reported implementation details	14
7 Main Reproducibility Gaps Left Unspecified by the Paper 14	14	7.1 Gap 1: Entity extraction and entity linking	14
.	15	7.2 Gap 2: Node types in the graphs	15
.	15	7.3 Gap 3: Construction rules for the DBpedia subgraph	15
. 15 7.4 Gap 4: Weighting of CKG edges	15	7.5 Gap 5: Entity feature initialization before R-GCN	15
6: How to form the set of dialogue entities E	15	7.6 Gap 6: How to form the set of dialogue entities E	15
representation extraction from DialoGPT	16	7.7 Gap 7: User representation extraction from DialoGPT	16
the recommendation label	16	7.8 Gap 8: Definition of the recommendation label	16
protocol	16	7.9 Gap 9: Splits and preprocessing protocol	16
.	16	7.10 Gap 10: Exact form of L_n in reranking	16
.	16	7.11 Gap 11: Exact prompt templates	16
.	16	7.12 Gap 12: Output parsing and ranking reconstruction	16
17	17		
8 A Complete Implementation Proposal to Fill the Gaps 17	17	8.1 Entity linking proposal	17
.	17	8.2 Node types for EKG and CKG	17
.	17	8.3 EKG construction proposal	17
8.4 CKG weighting proposal	18	8.5 Entity initialization proposal	18

8.6 Dialogue entity weighting proposal	18	8.7 User representation proposal	18
.	18	8.8 Supervision proposal	18
.	18	8.9 Data split proposal	18
.	19	8.10 Reranking label proposal	19
.	19	8.11 Prompt proposal for reranking	19
Prompt proposal for response generation	19	8.12 Evaluation protocol proposal	20
9 Adapting the Proposal from Llama2-7b to Qwen 3.5 20	20	9.1 What stays unchanged	20

.....	20	9.2 What changes	
.....	20	9.3 Recommended adaptation strategy	
21	9.4 Recommended model sizes		21
.....	21	9.5 Why structured outputs are preferable	
21	10 Implementation Blueprint: Modules and Pseudocode		21
.....	21	10.1 Suggested project structure	
.....	22	10.2 Recommended libraries	
.....	22	10.3 Pseudocode: data instance construction	
.....	22	10.4 Pseudocode: pre-ranking stage	
.....	23	10.5 Pseudocode: reranking with Qwen 3.5	
.....	23	10.6 Pseudocode: response generation	
.....	24	10.7 Pseudocode: semantic fusion module	
.....	24	10.8 Pseudocode: Qwen 3.5 soft-prompt reranker	
25	11 Suggested Prompt Designs for Qwen 3.5		25
.....	25	11.1 Reranking prompt	
.....	25	11.2 Response generation prompt	
26	12 Recommended Experimental Extensions for a Stronger Reimplementation		26
.....	26	12.1 Reranking formulation ablations	
.....	26	12.2 Graph ablations	
.....	26	12.3 Candidate serialization ablations	
.....	26	12.4 Adaptation ablations	
.....	27	12.5 Human evaluation protocol	
27			27
27	13 Overall Conclusion		27

TPLM for Conversational Recommender Systems Structured Technical Synthesis

1. Executive Summary

This document provides a coherent and structured synthesis of the proposal presented in the paper *A Two-Stage Prompt-Tuning Large Language Model as a Reranker for Conversational Recommender Systems*, integrating all the main points discussed previously:

- an executive summary of the proposal;
- a detailed step-by-step explanation of the complete pipeline;
- a mathematical formalization of the method;
- a precise description of the input elements, how they are obtained, and how they are transformed;
- a clear explanation of the experiments, datasets, preprocessing assumptions, baselines,

and metrics;

- a structured discussion of the implementation gaps left unspecified by the paper;
- a full implementation proposal to make the method reproducible;
- an adaptation strategy to replace Llama2-7b with Qwen 3.5;
- an implementation blueprint with pseudocode, PyTorch/Hugging Face modules, and prompt design.

The core idea of the paper is a two-stage coarse-to-fine recommendation framework for conversational recommender systems (CRS):

- 1) Pre-ranking stage: use conversational information and structured knowledge graphs to obtain a shortlist of candidate items.
- 2) Re-ranking stage: use a prompt-tuned large language model (LLM) to refine that candidate list by better exploiting nuanced conversational semantics.
- 3) Response generation stage: use the selected recommendation to generate a natural language response aligned with the conversation.

The proposal therefore combines three complementary information sources:

- textual conversational context,
- structured external and conversational knowledge,
- semantic and instruction-following capabilities of an LLM.

2. Problem Setting and Overall Objective

The paper addresses a Conversational Recommender System (CRS) setting in which a model must jointly solve:

- 1) Recommendation: identify the most suitable item for the user given the dialogue history.

4

TPLM for Conversational Recommender Systems Structured Technical Synthesis

- 2) Conversation generation: produce a fluent and coherent system response that naturally incorporates the recommendation.

Let the dialogue context be denoted by

$$C = \{s_t\}_{t=1}^n,$$

where s_t is the utterance at turn t . Each utterance is itself a sequence of words:

$$s_t = \{w_{jt}\}_{j=1}^{m_t}$$

$$j=1, w_j \in V,$$

where V is the vocabulary.

At recommendation time, the model takes the current dialogue context C and aims to:

- select a recommendation item I_t ,
- generate a response R_t that includes I_t coherently.

The main difficulty is that a strong CRS must capture both:

- global and structured knowledge about items and entities,
- fine-grained semantic details in the dialogue, such as subtle preferences, exclusions, temporal constraints, and implicit hints.

The paper argues that previous systems tend to emphasize one of these aspects more than the other, and proposes a two-stage model to leverage both.

3. High-Level Summary of the Proposed Method

The method, denoted as TPLM (Two-stage Prompt-tuning Large language Model), consists of three main stages.

3.1. Stage 1: Pre-ranking

The first stage builds a preliminary shortlist of candidate items using:

- the dialogue history,
- an External Knowledge Graph (EKG) derived from DBpedia,
- a Conversational Knowledge Graph (CKG) built from entity co-occurrence in dialogues.

This stage encodes both textual and structured information and produces a *coarse* ranking, typically selecting the top- c candidates (in the paper, $c = 50$).

3.2. Stage 2: Re-ranking with Prompt-Tuned LLM

The second stage takes:

- the dialogue context,
- the shortlist produced by Stage 1,

and uses a prompt-tuned LLM to refine the ranking.

The purpose is to inject a more precise semantic understanding of user intentions and conversational constraints into the final recommendation decision.

3.3. Stage 3: Response Generation

Finally, once the re-ranked top item is selected, the LLM generates a conversational response that:

- sounds natural,
- is consistent with the dialogue,
- explicitly or implicitly justifies the recommendation,
- remains aligned with the selected item.

4. Complete Pipeline: Step-by-Step Explanation

This section explains the complete pipeline in a clear operational order.

4.1. Step 1: Receive the dialogue history

The system starts from a multi-turn dialogue history:

$$C = \{s_1, s_2, \dots, s_n\}.$$

This history includes all previous turns exchanged between the user and the system. It is the primary raw input used for both recommendation and generation.

4.2. Step 2: Extract entities from the dialogue

Relevant entities are identified from the conversation. In the movie recommendation domain, these may include:

- movie titles,
- genres,
- actors,
- directors,

- temporal clues such as decades or periods.

These entities are later used to construct or query knowledge graphs.

4.3. Step 3: Build the External Knowledge Graph (EKG)

From the linked entities extracted from the dataset, the model constructs a subgraph from

DBpedia:

$$G_E = (E, R, T),$$

where:

- E is the set of entities,
- R is the set of relation types,
- T is the set of triples $\langle e_1, r, e_2 \rangle$.

This graph captures structured world knowledge.

4.4. Step 4: Build the Conversational Knowledge Graph (CKG)

A second graph is constructed directly from the dataset dialogues. Two entities are connected if they co-occur in the same dialogue:

$$G_C = (E, A),$$

where A denotes the set of co-occurrence edges.

This graph captures conversational patterns specific to the recommendation domain.

4.5. Step 5: Encode the dialogue text with RoBERTa

The conversation text is encoded into contextualized word embeddings using

$$\text{RoBERTa: } T = [\alpha_1; \alpha_2; \dots; \alpha_{n_w}] \in \mathbb{R}^{n_w \times d},$$

where:

- n_w is the number of tokens in the dialogue history,
- d is the embedding dimension.

This matrix captures the semantic representation of the dialogue.

4.6. Step 6: Encode the external KG with R-GCN

The EKG is encoded with a relational graph convolutional network (R-GCN). The update for entity e at layer $l + 1$ is:

$$h_e^{(l+1)} = \sigma \left(\frac{1}{Z_{e,r}} \sum_{e' \in E_e^r} W^{(l)} e' + W^{(l)} h_e^{(l)} \right),$$

TPLM for Conversational Recommender Systems Structured Technical Synthesis

where:

- E_e^r is the set of neighbors of e under relation r ,
- $Z_{e,r}$ is a normalization factor,
- $W^{(l)}$

r is a relation-specific weight matrix,

- $W^{(l)}$ is the self-loop transformation,
- σ is a nonlinearity, typically ReLU.

This yields a matrix of entity embeddings:

$$M = [\beta_1; \beta_2; \dots; \beta_{ne}] \in \mathbb{R}_e^{n \times d}.$$

4.7. Step 7: Encode the conversational KG with GCN

The CKG is encoded using a standard graph convolutional network

$$(\text{GCN}): H^{(l+1)} = \sigma D^{-1/2} \tilde{A} D^{-1/2} H^{(l)} W^{(l)},$$

with

$$\tilde{A} = A + I,$$

where:

- A is the adjacency matrix of the CKG,
- I is the identity matrix,
- $\tilde{D}$ is the degree matrix of $\tilde{A}$.

The initial node features are set as:

$$H^{(0)} = M.$$

Thus, the CKG is initialized using the R-GCN entity embeddings learned from the EKG.

The output is:

$$M = [\gamma_1; \gamma_2; \dots; \gamma_{ne}] \in \mathbb{R}_e^{n \times d}.$$

4.8. Step 8: Perform semantic fusion between dialogue and entities

The paper notes that the text representation and the graph-based entity representation lie in different semantic spaces. To bridge this gap, the model introduces a bilinear fusion mechanism.

Let E be the matrix of entity embeddings corresponding to entities mentioned in the dialogue. Then:

$$F = T^T W E,$$

and the enriched text representation is:

$$\tilde{T} = T + EF.$$

Operationally, this step injects knowledge-graph information into the textual representation of

the dialogue.

4.9. Step 9: Compute a preliminary recommendation with DialoGPT The fused text

representation $\tilde{T}$ is fed into DialoGPT to obtain a user representation: $h_u =$

$$\text{DialoGPT}(\tilde{T}).$$

Then, for each candidate entity e_k , the model computes:

$$P_{\text{rec}}(e_k) = \text{Softmax}(h_u \cdot h_{e_k}),$$

where h_{e_k} is the graph-enhanced embedding of candidate entity e_k .

The recommendation loss is:

$$L_{\text{rec}} = -\sum_{k=1}^n y_k \log P_{\text{rec}}(e_k),$$

where $y_k \in \{0, 1\}$ indicates whether e_k is a target item.

After training, the model selects the top- c candidates:

$$I^c = \text{TopK}(P_{\text{rec}}, c),$$

with $c = 50$ in the original paper.

4.10. Step 10: Re-rank candidates with a prompt-tuned LLM The

second stage takes:

- the dialogue context C_n ,
- the candidate list I_n^c ,
- a prompt template $P re_c$,
- a label L_n .

The reranking input is constructed as:

$$B_n = [C_n; I_n^c],$$

$$A_n = [P re_c; B_n].$$

After tokenization, the paper conceptually distinguishes:

- prompt tokens Z_1 ,
- context+candidate tokens Z_2 ,
- label tokens Z_3 .

Prompt embeddings and model embeddings are then processed by the LLM. The prompt-tuning loss is formulated autoregressively as:

$$L_{\text{pro}} = -\sum_{m=y_2+1}^{y_3} \log P(p_{<m}),$$

where the model predicts the target label tokens conditioned on the prompt and the input. Only the prompt-related parameters and the output layer are updated, while the rest of the LLM remains frozen.

4.11. Step 11: Generate the final response

Once the final reranked recommendation I_t is selected, the system generates a natural

$$\text{response: } R = \text{LLM}(P\text{ re}_s, C_{t-1}, I_t),$$

where:

- $P\text{ re}_s$ is a response-generation prompt,
- C_{t-1} is the dialogue history,
- I_t is the final recommendation.

5. Input Elements, Their Origin, and Their Transformation This section summarizes the input elements required by the model and how they are obtained.

5.1. Dialogue history

Origin: the raw dataset dialogue turns.

Transformation:

- tokenize dialogue,
- preserve turn structure,
- optionally truncate history to fit model length constraints,
- encode with RoBERTa.

5.2. Dialogue entities

Origin: extracted from dialogue text.

Transformation:

- entity mention detection,
- entity linking or normalization,

10

TPLM for Conversational Recommender Systems Structured Technical Synthesis

- use linked entities to query DBpedia and build graph nodes,
- use mentions for CKG construction.

5.3. External KG

Origin: DBpedia subgraph built from linked entities.

Transformation:

- collect relevant triples,
- build relational graph,
- encode with R-GCN.

5.4. Conversational KG

Origin: entity co-occurrence across dialogues.

Transformation:

- connect entities that co-occur in the same dialogue,
- optionally weight edges by frequency,
- encode with GCN initialized by EKG embeddings.

5.5. Candidate shortlist

Origin: pre-ranking output.

Transformation:

- compute scores for all candidate items,
- select top-c,
- serialize them for reranking.

6. Experiments: Datasets, Evaluation, and Baselines

6.1. Datasets

The paper evaluates on two public conversational recommendation datasets in the movie domain.

6.1.1. *INSPIRED*

A relatively small English conversational recommendation dataset

- with:
- 1,001 dialogues,
 - 35,811 utterances,
 - 1,783 items.

11

TPLM for Conversational Recommender Systems Structured Technical Synthesis

6.1.2. *ReDial*

A larger English movie conversational recommendation dataset with:

- 10,006 dialogues,
- 182,150 utterances,
- 51,699 items.

6.2. What the paper reports clearly

The paper reports:

- datasets,
- recommendation metrics,
- dialogue generation metrics,
- implementation frameworks,
- certain hyperparameters.

6.3. What the paper does not specify clearly

However, the paper does not explicitly define:

- exact train/validation/test splits,
- how dialogue instances are formed per turn,
- target extraction when multiple items appear,
- exact entity linking pipeline,
- exact prompt templates,
- exact mapping from generated reranking text to ranked items,
- exact truncation policy for long dialogues.

These omissions are important for reproducibility.

6.4. Baselines

The baselines can be grouped into three families.

6.4.1. *Traditional recommendation baselines*

- Popularity,
- TextCNN.

12

TPLM for Conversational Recommender Systems Structured Technical Synthesis

6.4.2. *LLM / general conversational baselines*

- Transformer,
- Llama2-7b,
- GPT-3.5-turbo.

6.4.3. *Conversational recommender baselines*

- ReDial,
- KBRD,
- KGSF,
- C²-CRS,
- UniCRS,
- KERL,
- MKCRS.

6.5. Evaluation metrics

6.5.1. *Recommendation metrics*

- Recall@1,
- Recall@10,
- Recall@50.

6.5.2. *Dialogue generation metrics*

- Distinct-2,
- Distinct-3,
- Distinct-4.

6.5.3. Human evaluation

The paper also reports human judgments on:

- Fluency,
- Informativeness,

with a 0–2 rating scale.

6.6. Reported implementation details

The paper states:

- PyTorch framework,
- RoBERTa embedding size = 768,
- one-layer R-GCN,
- one-layer GCN,
- DialoGPT embedding size = 768,
- candidate number $c = 50$,
- pre-ranking optimizer: AdamW,
- pre-ranking learning rate = 0.001,
- pre-ranking batch size = 32,
- reranking backbone: Llama2-7b,
- reranking batch size = 128,
- reranking learning rate = 0.001.

7. Main Reproducibility Gaps Left Unspecified by the Paper

A major part of the previous analysis focused on the fact that the paper gives a convincing high-level architecture but leaves many implementation-critical details underspecified.

7.1. Gap 1: Entity extraction and entity linking

The paper says that entities are extracted from dialogues and used to construct the EKG and CKG, but it does not define:

- mention detection,

- title normalization,
- disambiguation strategy,
- alias handling,
- failure cases when DBpedia has no reliable link.

7.2. Gap 2: Node types in the graphs

The paper does not specify whether the graphs only contain movies or also include:

- genres,
- actors,
- directors,
- temporal abstractions such as decades.

7.3. Gap 3: Construction rules for the DBpedia subgraph

The paper does not define:

- graph expansion depth,
- relation filtering,
- node pruning,
- degree caps,
- whether edge directions are preserved.

7.4. Gap 4: Weighting of CKG edges

The paper only says that two entities are connected if they co-occur in a dialogue. It does not specify whether edges are:

- binary,
- frequency-based,
- PMI-based,
- recency-sensitive.

7.5. Gap 5: Entity feature initialization before R-GCN

The paper defines the graph propagation layers but not the entity initialization strategy.

7.6. Gap 6: How to form the set of dialogue entities E

The paper states that the embeddings of entities mentioned in the dialogue are used in semantic fusion, but it does not specify:

- whether all history turns are included,
- whether duplicate mentions are collapsed,
- whether recent mentions are weighted more.

7.7. Gap 7: User representation extraction from DialoGPT

The paper writes $h_u = \text{DialoGPT}(\tilde{T})$, but not whether:

- the last token state is used,
- mean pooling is used,
- an EOS state is used,
- an additional projection is used.

7.8. Gap 8: Definition of the recommendation label

The paper uses $y_k \in \{0, 1\}$ but does not specify the exact supervision

- signal:
- first recommended movie,
 - all recommended movies in a response,
 - per-dialogue vs per-turn labeling.

7.9. Gap 9: Splits and preprocessing protocol

The paper does not fully detail:

- dataset split strategy,
- truncation,
- multi-target handling,
- candidate universe restrictions,
- dialogue instance generation.

7.10. Gap 10: Exact form of L_n in reranking

The paper introduces L_n as the label sequence but does not specify whether it

- is:
- a movie title,
 - a candidate ID,
 - a complete ranking,
 - a free-form answer.

7.11. Gap 11: Exact prompt templates

The paper does not formally publish:

- the reranking prompt $P re_c$,
- the response-generation prompt $P re_s$.

16

TPLM for Conversational Recommender Systems Structured Technical Synthesis

7.12. Gap 12: Output parsing and ranking reconstruction

The reranker is treated in a generative manner, but the paper does not explain how generated text is transformed into a reliable ranking decision.

8. A Complete Implementation Proposal to Fill the Gaps

This section proposes a coherent implementation design to make the method reproducible.

8.1. Entity linking proposal

Use a four-step pipeline:

- 1) text normalization,
- 2) mention extraction using dataset annotations if available, otherwise dictionary/fuzzy matching,
- 3) entity disambiguation,
- 4) fallback local entities for CKG-only use.

A practical disambiguation function is:

$$s(e \mid m, C) = \lambda_1 s_{\text{title}} + \lambda_2 s_{\text{year}} + \lambda_3 s_{\text{context}} + \lambda_4 s_{\text{prior}},$$

and choose the entity with highest score if the score exceeds a threshold τ .

8.2. Node types for EKG and CKG

Define a heterogeneous graph including:

- movie nodes,
- genre nodes,
- actor nodes,
- director nodes,
- decade nodes.

8.3. EKG construction proposal

Use:

- seed nodes: all linked entities from the dataset,
- 1-hop expansion for most relations,
- selective 2-hop expansion only for highly informative relations,

17

TPLM for Conversational Recommender Systems Structured Technical Synthesis

- relations such as starring, director, genre, writer, producer, releaseDate, •
transform release date into decade abstractions,
- cap degree to reduce noise.

8.4. CKG weighting proposal

Instead of a binary graph, define:

$$A_{ij} = \log(1 + \text{cooc}(e_i, e_j)),$$

or a normalized PMI-like weighting if desired.

8.5. Entity initialization proposal

Initialize entities using textual metadata:

$$x_e = \text{RoBERTa}(\text{name}(e) \oplus \text{type}(e) \oplus \text{description}(e)).$$

If no description is available, use:

$$x_e = \text{RoBERTa}(\text{name}(e) \oplus \text{type}(e)).$$

8.6. Dialogue entity weighting proposal

Let:

$$E_C = \{e \in E : e \text{ appears in } C\}.$$

Define recency-aware weights:

$$\omega_e = X_{t=1}^T e^{-t},$$

$$1[e \in s_t] \alpha^T$$

with $\alpha \in (0, 1]$, and weight entity embeddings accordingly.

8.7. User representation proposal

Use the last EOS hidden state from DialoGPT:

$$h_u = W_u h_{\text{EOS}} + b_u.$$

8.8. Supervision proposal

For each recommendation instance:

- input: dialogue history up to turn $t - 1$,
- recommendation target: the main recommended movie in system turn t ,
- dialogue target: full system utterance at turn t .

18

TPLM for Conversational Recommender Systems Structured Technical Synthesis

8.9. Data split proposal

If official splits are unavailable or unclear, use:

80% train, 10% validation, 10% test,

splitting by complete dialogues only.

8.10. Reranking label proposal

Use structured output labels:

$$L_n = \text{ANSWER: } \langle \text{candidate_id} \rangle.$$

This is preferable to raw movie-title generation because it:

- avoids tokenization ambiguity,
- avoids confusion among similar titles,
- simplifies parsing.

8.11. Prompt proposal for reranking

A concrete reranking prompt can be:

You are a conversational movie recommender.

Given the dialogue history and a candidate list, choose the single best movie. Pay special attention to explicit preferences such as genre, age/decade, actors, directors, and exclusions.

Return only:

ANSWER: <id>

[Dialogue History]

{history}

[Candidate List]

1. Halloween (1978) | genres: Horror | decade: 1970s

2. The Shining (1980) | genres: Horror | decade: 1980s

...

50. Ouija (2014) | genres: Horror | decade: 2010s

8.12. Prompt proposal for response generation

A concrete response-generation prompt can be:

You are a helpful and natural conversational movie recommender.

Use the dialogue history and the selected recommendation to produce one concise response.

The response must:

- sound natural,
- mention the selected movie,

19

TPLM for Conversational Recommender Systems Structured Technical Synthesis

- justify it using the user's preferences,
- avoid mentioning any movie not selected.

[Dialogue History]

{history}

[Selected Recommendation]

Halloween (1978) | genres: Horror | decade: 1970s | reason hints: classic, old horror [Output]

8.13. Evaluation protocol proposal

For more reproducible human evaluation:

- sample 200 test instances per dataset,
- use 3 annotators,
- rate fluency and informativeness,
- report average scores and inter-annotator agreement.

9. Adapting the Proposal from Llama2-7b to Qwen 3.5

The original paper uses Llama2-7b in the reranking stage and in response generation. A natural extension is to replace it with Qwen 3.5.

9.1. What stays unchanged

The following blocks can remain conceptually identical:

- RoBERTa dialogue encoder,
- EKG construction,
- R-GCN,
- CKG construction,
- GCN,
- semantic fusion,
- DialoGPT-based pre-ranking.

9.2. What changes

The LLM-dependent parts that change are:

- tokenizer,

20

TPLM for Conversational Recommender Systems Structured Technical Synthesis

- soft-prompt implementation,
- hidden dimension of the LLM,
- prompt serialization format,
- output parsing.

9.3. Recommended adaptation strategy

A practical migration path is:

- 1) replace Llama tokenizer by Qwen tokenizer,
- 2) keep the LLM frozen,
- 3) train only a soft prompt (and optionally an output head),
- 4) use structured labels such as ANSWER: 17,
- 5) preserve the same candidate shortlist size ($c = 50$),
- 6) keep the pre-ranking stage unchanged,
- 7) use Qwen also for response generation.

9.4. Recommended model sizes

For research prototyping:

- use a mid-size Qwen 3.5 model for serious experiments,
- use a smaller Qwen 3.5 checkpoint for debugging and ablation prototyping.

9.5. Why structured outputs are preferable

For Qwen-based reranking, structured IDs are more robust than title generation:

- they simplify decoding,
- reduce title ambiguity,
- allow deterministic evaluation,
- better match a shortlist-based reranking problem.

10. Implementation Blueprint: Modules and Pseudocode

10.1. Suggested project structure

```

data/
  raw_dialogues/
  processed/
  kg/
  splits/

src/
  preprocessing/
    normalize.py
    entity_linking.py
    build_ekg.py
    build_ckg.py
    serialize_candidates.py
  models/
    roberta_encoder.py
    rgcn_encoder.py
    ckg_gcn.py
    semantic_fusion.py
    preranker.py
    qwen_soft_prompt_reranker.py
    response_generator.py
  training/
    train_preranker.py
    train_reranker.py
    eval_recommendation.py
    eval_dialogue.py
  inference/
    pipeline.py

```

10.2. Recommended libraries

- PyTorch for model training,
- Hugging Face Transformers for RoBERTa, DialoGPT, and Qwen,
- PyTorch Geometric or DGL for graph models,
- optionally PyTorch Lightning for cleaner training orchestration.

10.3. Pseudocode: data instance construction

```

def build_instance(dialogue, turn_t):
    history = get_history(dialogue, until_turn=turn_t - 1)
    target_item = extract_target_recommendation(dialogue[turn_t])
    target_response = dialogue[turn_t].text

    mentions = entity_link(history)
    ekg_nodes, ekg_edges = build_or_lookup_ekg_subgraph(mentions)
    ckg_nodes, ckg_edges = build_or_lookup_ckg_subgraph(dialogue.id)

    return {
        "history": history,

```

```

    "target_item": target_item,
    "target_response": target_response,
    "mentions": mentions,
    "ekg": (ekg_nodes, ekg_edges),
    "ckg": (ckg_nodes, ckg_edges),
}

```

10.4. Pseudocode: pre-ranking stage

```

def pre_rank(instance, roberta, rgcn, gcn, fusion, dialogpt, item_bank, top_c=50): T =
roberta_encode_history(instance["history"], roberta) # [nw, d] M = rgcn_encode(instance["ekg"],
rgcn) # [ne, d] M_enh = gcn_encode(instance["ckg"], M, gcn) # [ne, d] E =
gather_dialogue_entity_embeddings(instance["mentions"], M_enh) # [k, d]

T_fused = fusion(T, E) # [nw, d] h_u = dialogpt_user_embedding(T_fused, dialogpt) # [d]

scores = item_bank @ h_u # [num_items ]
candidate_ids = topk(scores, k=top_c)

return candidate_ids, scores[candidate_ids]

```

10.5. Pseudocode: reranking with Qwen 3.5

```

def rerank_with_qwen(history, candidate_list, qwen_tokenizer, qwen_model, soft_prompt) :
    prompt = build_rerank_prompt(history, candidate_list)
    inputs = qwen_tokenizer(prompt, return_tensors="pt", truncation=True)

    outputs = qwen_generate_with_soft_prompt(
        model=qwen_model,
        tokenizer=qwen_tokenizer,
        inputs=inputs,
        soft_prompt=soft_prompt,
        max_new_tokens=8,
    )

    pred_id = parse_answer_id(outputs.text)
    if pred_id is None or pred_id not in candidate_list.local_ids:
        pred_id = candidate_list.local_ids[0]

    return pred_id

```

10.6. Pseudocode: response generation

```

def generate_response(history, selected_item, qwen_tokenizer, qwen_model): prompt =
    build_response_prompt(history, selected_item)
    inputs = qwen_tokenizer(prompt, return_tensors="pt", truncation=True) 23

```



```

outputs = qwen_model.generate(**inputs, max_new_tokens=80)
return qwen_tokenizer.decode(outputs[0], skip_special_tokens=True)

```

10.7. Pseudocode: semantic fusion module

```

class BilinearSemanticFusion(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.W = nn.Parameter(torch.randn(d_model, d_model) * 0.02)

    def forward(self, T, E):
        # T: [nw, d], E: [k, d]
        F = T @ self.W @ E.transpose(0, 1) # [nw, k]
        T_enh = T + F @ E # [nw, d]
        return T_enh

```

10.8. Pseudocode: Qwen 3.5 soft-prompt reranker

```

class SoftPromptQwenReranker(nn.Module):
    def __init__(self, model_name, n_soft=32):
        super().__init__()
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            torch_dtype="auto",
            device_map="auto",
        )
        for p in self.model.parameters():
            p.requires_grad = False

        hidden = self.model.config.hidden_size
        self.soft_prompt = nn.Parameter(torch.randn(n_soft, hidden) * 0.02)

    def forward(self, input_ids, attention_mask, labels=None):
        tok_emb = self.model.get_input_embeddings()(input_ids)
        bsz = tok_emb.size(0)

        soft = self.soft_prompt.unsqueeze(0).expand(bsz, -1, -1)
        inputs_embeds = torch.cat([soft, tok_emb], dim=1)

        soft_mask = torch.ones(
            bsz, self.soft_prompt.size(0),
            dtype=attention_mask.dtype,
            device=attention_mask.device
        )
        attention_mask = torch.cat([soft_mask, attention_mask], dim=1)

        if labels is not None:
            pad = torch.full(
                (bsz, self.soft_prompt.size(0)),
                -100,

```

```

        dtype=labels.dtype,
        device=labels.device
    )
    labels = torch.cat([pad, labels], dim=1)

    return self.model(
        inputs_embeds=inputs_embeds,
        attention_mask=attention_mask,
        labels=labels,
    )

```

11. Suggested Prompt Designs for Qwen 3.5

11.1. Reranking prompt

System:
You are a movie recommendation reranker.

User:
Select the single best movie from the candidate list based on the dialogue. Follow explicit user constraints strictly.
Prefer candidates that match genre, year/era, and semantic clues from the conversation .
Return only:
ANSWER: <candidate_id>

Dialogue:
User: I am looking for a horror film.
System: Did you see A Nightmare on Elm Street? What movie are you planning to watch this weekend?
User: I have seen it. I plan to watch some old movies.

Candidates:
1. It (2017) | Horror | 2010s
2. The Shining (1980) | Horror | 1980s
3. Carrie (1976) | Horror | 1970s
4. Halloween (1978) | Horror | 1970s
...
50. Ouija (2014) | Horror | 2010s

11.2. Response generation prompt

System:
You are a helpful and natural movie recommender.

User:
Write one conversational response using the selected recommendation. The reply must:

- mention the movie naturally,
- justify it with the user's preferences,

- sound concise and friendly,

- avoid mentioning unselected candidates.

Dialogue:

User: I am looking for a horror film.

System: Did you see A Nightmare on Elm Street? What movie are you planning to watch this weekend?

User: I have seen it. I plan to watch some old movies.

Selected recommendation:

Halloween (1978) | Horror | 1970s

12. Recommended Experimental Extensions for a Stronger Reimplementation

To strengthen the implementation and provide a more convincing Qwen-based reproduction, the following additional experiments are recommended.

12.1. Reranking formulation ablations

Compare:

- free-form title generation,
- structured ID generation,
- discriminative classification over the 50 candidates.

12.2. Graph ablations

Compare:

- binary CKG vs weighted CKG,
- EKG-only vs EKG+CKG,
- movie-only graphs vs heterogeneous graphs with genres/actors/directors.

12.3. Candidate serialization ablations

Compare:

- title-only candidates,
- title + year,
- title + year + genre + decade,
- title + year + genre + decade + director.

12.4. Adaptation ablations

Compare:

- soft-prompt only,
- soft-prompt + LoRA,
- soft-prompt + classifier head.

12.5. Human evaluation protocol

In addition to mean scores, report:

- inter-annotator agreement,
- number of evaluated samples,
- annotation guidelines.

13. Overall Conclusion

The proposal in the paper is strong at the architectural level because it

- combines:
- structured external knowledge,
 - conversational co-occurrence knowledge,
 - semantic dialogue understanding,
 - LLM-based instruction following.

Formally, the full pipeline can be summarized as:

$$C \rightarrow (T, G_E, G_C) \rightarrow (M, M) \rightarrow \tilde{T} \rightarrow h_u \rightarrow l^c \rightarrow \text{LLM-rerank} \rightarrow l_t \rightarrow \text{LLM-generate} \rightarrow R.$$

However, a faithful and reproducible implementation requires making explicit several critical design choices not fully specified in the paper, especially:

- entity extraction and linking,
- graph construction details,
- recommendation label definition,
- reranking label format,
- prompt design,
- output parsing,
- dataset split protocol.

Once these gaps are resolved, the architecture can be implemented in a coherent manner and adapted naturally from Llama2-7b to Qwen 3.5 with minimal changes to the second-stage reranker and the response generation module.